Proposal for CrisisEcho: LLM-Augmented Early Detection of Local Emergencies from Social Media

Samuel Enam Zih
Computer Science
Virginia Tech
Alexandria, VA, USA
szih@vt.edu
(MERCIFUL LEADER)

Christopher Drake Williams
Computer Science
Virginia Tech
Alexandria, VA, USA
cdwilliams@vt.edu

Arjun Sivakumar
Computer Science
Virginia Tech
Alexandria, VA, USA
arjuns@vt.edu

Ki Hong Park
Computer Science
Virginia Tech
Alexandria, VA, USA
kihong@vt.edu

Shiyi Liu
Computer Science
Virginia Tech
Alexandria, VA, USA
shiyiliu@vt.edu

***Abstract*—This project proposes CrisisEcho, a consumer-facing application that leverages Retrieval-Augmented Generation (RAG) and large language model (LLM) agents to detect, classify, and summarize hyperlocal emergencies from multi-source social media streams in near real-time. Traditional emergency response systems rely almost exclusively on official channels such as 911 dispatch, government sensors, and police scanners, which frequently lag actual events by minutes or hours. CrisisEcho addresses this gap by ingesting posts from Twitter/X, Reddit, Bluesky, Mastodon, Nextdoor, and Telegram alongside corroborating signals from official feeds including the National Weather Service (NWS), USGS Earthquake Hazards, FEMA OpenFEMA, PulsePoint, and GDELT. The system stores processed posts in MongoDB Atlas using native 2dsphere geospatial indexing, and uses a Pinecone-hosted vector index for payload-filtered semantic retrieval. A three-step LLM agent chain performs event clustering, severity scoring, and natural-language alert generation. Results are delivered through a React web application featuring a Leaflet.js interactive map with color-coded crisis hotspots, a pin detail panel, a WebSocket real-time notification engine, and a natural-language query interface. The backend is built in Go for high-throughput concurrent alert delivery, while the AI pipeline runs in Python using LangChain and Sentence-Transformers. The system is fully containerized with Docker Compose and deployed on GCP Cloud Run. This project demonstrates how LLMs transform relational and document stores into open-world, semantically reasoning systems capable of detecting crises from noisy, unstructured, human-generated text.**

***Index Terms*—emergency detection, social media mining, retrieval-augmented generation, large language models, geospatial databases, MongoDB, real-time systems, crisis informatics, hyperlocal awareness**

I. Introduction

A. Motivation

Traditional emergency response systems rely almost entirely on official channels, including 911 calls, government sensors, and police scanners. These channels frequently arrive minutes or even hours after a crisis begins. In fast-moving events such as flash floods, active shooter situations, widespread power outages, or natural disasters, those delays can have life-threatening consequences.

In today's connected world, ordinary people are frequently the first witnesses, posting real-time signals on social media before any official report is filed. Real-world examples of such posts include statements such as "Water rising fast on my street in Leesburg — cars already stuck!", "Heard multiple gunshots near Leesburg Outlet Mall, everyone running", "My basement is turning into a pool lmao — serious flooding on Maple Ave", and "Sounds like fireworks but way too many — near the mall." These posts contain the earliest, most hyperlocal clues about emerging crises, typically appearing before official reports are issued.

B. Problem Statement

Existing monitoring tools, both commercial and research prototypes, are predominantly keyword-based. They scan for exact terms such as "flood" or "gunshots." This approach has three critical failure modes. First, it misses nuance and context, such as indirect crisis signals expressed in colloquial or figurative language. Second, it struggles with sarcasm, slang, and regional expressions. Third, it generates both false negatives, missing real events, and false positives, flooding operators with noise, making such systems unreliable for timely action.

Furthermore, most existing crisis monitoring tools are designed for professional emergency management operators, not everyday citizens. There is no widely accessible consumer application that synthesizes multi-source social signals, applies LLM-based semantic reasoning, and delivers plain-language, map-based crisis alerts to the general public.

C. Significance

CrisisEcho directly demonstrates how large language models are transforming database systems, as highlighted in the course material. The system instantiates four key paradigm shifts. In the dimension of Text to Semantics, semantic understanding capabilities allow the system to discern nuances, context, and subtleties that traditional keyword-based data

management cannot. In the dimension of Retrieval to Reasoning, multi-step LLM agent chains perform event clustering, severity assessment, and alert generation far beyond simple lookup retrieval. In the dimension of Vertical Domains to Multiple Domains, the pipeline is extensible from crisis detection to positive community event detection without schema changes. In the dimension of Closed World to Open World, the system generalizes to evolving, unstructured real-world language without rigid schemas or vocabulary lists.

By building CrisisEcho as a consumer-facing app, the project directly puts earlier, more intelligent crisis awareness in the hands of everyday people, supporting faster community coordination and personal safety decision-making.

II. Literature Review: Social Media Crisis Detection

A. Social Media as a Crisis Sensor

Social media platforms have been extensively studied as real-time data sources for crisis detection and humanitarian response. Imran et al. [1] demonstrated that Twitter contains actionable crisis-related content during disasters and developed large-scale annotated corpora (CrisisNLP, HumAID) to train classification models at scale. Olteanu et al. [2] introduced CrisisLex, a lexicon for collecting and filtering microblogged communications during crises, establishing a foundational vocabulary approach that CrisisEcho supersedes with semantic embeddings.

Middleton et al. [4] demonstrated real-time crisis mapping from social media, showing that geographically tagged social posts correlate strongly with ground-truth disaster events. Kryvasheyeu et al. [5] showed that social media activity can provide rapid damage assessment during natural disasters, validating social signals as a credible early-warning source.

B. Limitations of Keyword-Based Approaches

The primary limitation of existing systems is their reliance on fixed vocabulary matching. Such approaches consistently fail to detect crisis signals expressed through indirect language, regional idioms, or evolving slang. Alam et al. [3] established benchmarks showing that even fine-tuned classification models trained on labeled crisis data struggle to generalize to novel crisis types or geographic regions, motivating the open-world approach taken by CrisisEcho.

C. Machine Learning in Crisis Informatics

Nguyen et al. [9] proposed convolutional neural networks (CNNs) for identifying valuable crisis-related information from tweets, demonstrating improvement over keyword baselines. More recently, transformer-based models have become the dominant architecture. Alam et al. [3] showed that BERT-based classifiers trained on HumAID achieve strong F1 scores across multiple crisis event types and classification tasks including crisis type identification, humanitarian category labeling, and informativeness scoring.

CrisisEcho extends this line of work by replacing single-step classification with a multi-step LLM agent reasoning chain, enabling not only detection but also event clustering, severity scoring, and actionable alert generation in a single unified pipeline.

III. Literature Review: LLMs, RAG, and Geospatial Database Systems

A. Retrieval-Augmented Generation

Lewis et al. [6] introduced Retrieval-Augmented Generation (RAG) as a framework for combining dense retrieval with generative language models, enabling models to ground their outputs in retrieved evidence. This approach is central to CrisisEcho: rather than relying on LLM parametric knowledge alone, the agent retrieves semantically similar recent posts as context before generating a crisis assessment. This grounding reduces hallucination and anchors the LLM output to real, timestamped evidence.

B. LLM Agents and Multi-Step Reasoning

Recent work on LLM agents, including the ReAct framework [7] and LangChain orchestration, has demonstrated that chaining multiple LLM calls with intermediate reasoning steps significantly outperforms single-prompt approaches on complex analytical tasks. CrisisEcho applies a three-step agent chain: event clustering, severity scoring with rationale, and natural-language alert generation. This decomposition mirrors the cognitive process of a human emergency analyst while operating at machine speed.

C. Geospatial Database Systems

MongoDB's native 2dsphere indexing supports GeoJSON geometries and enables efficient geospatial queries including `$near` (posts within radius of a point), `$geoWithin` (posts inside a polygon), and `$geoIntersects` (overlapping geometries). This eliminates the need for PostGIS as a separate extension and simplifies the deployment stack. Geospatial indexing in document databases has been shown to scale efficiently to millions of geotagged records with sub-100ms query latency [11], meeting the real-time requirements of CrisisEcho.

D. Vector Search and Hybrid Retrieval

Karpukhin et al. [8] demonstrated that dense passage retrieval using bi-encoder models significantly outperforms BM25 keyword retrieval for open-domain question answering, motivating the use of dense vector search in CrisisEcho. Pinecone's hosted vector index provides Approximate Nearest Neighbor (ANN) search via HNSW with metadata filtering, enabling combined vector similarity plus geographic bounding box queries in a single retrieval step. Unlike other free-tier vector database options, Pinecone's Starter plan provides persistent storage with no inactivity-based deletion, making it appropriate for a semester-long course project.

IV. Literature Review: Geolocation from Unstructured Text

A. Text-Based Geocoding

A fundamental challenge in social media crisis detection is that fewer than 2% of tweets carry GPS coordinates [4]. The Carmen geocoder [10] addresses this by extracting location references from post text, user profile location fields, and time zone metadata, achieving reasonable precision on English-language social media. spaCy's Named Entity Recognition (NER) pipeline provides a complementary approach by identifying place names, neighborhood references, and intersection descriptions in post text.

CrisisEcho implements a cascading geocoding strategy: GPS metadata (when available) takes highest priority, followed by Carmen extraction, spaCy NER, user profile location, and finally a batch inference step that uses nearby geotagged posts as geographic priors for posts with no extractable location.

V. Dataset Description

CrisisEcho uses a multi-layered data collection strategy spanning social media platforms, official government and emergency APIs, crowdsourced citizen platforms, and labeled research datasets. All sources are either publicly available APIs, academic datasets compliant with their respective terms of use, or public-facing social media streams accessible under standard research guidelines.

A. Social Media Platforms — Live Streaming

Twitter/X is accessed via Tweepy filtered stream for live ingestion and supplemented by historical academic datasets (CrisisNLP, HumAID, Kaggle Disaster Tweets) for training and evaluation. Twitter/X provides broad real-time crisis signals and is the most studied platform in crisis informatics.

Reddit is accessed via PRAW streaming from relevant local subreddits (r/Virginia, r/nova, r/news, r/Alexandria, r/leesburg) and supplemented by Pushshift historical dumps. Reddit posts are longer-form and often verified by community upvotes, making them a high-value source for geographically attributed crisis reports.

Bluesky is accessed through the AT Protocol public firehose, which imposes no rate limits and provides a fully open stream of public posts. Bluesky is a growing platform with active local community usage and represents a rate-limit-free alternative to Twitter/X.

Mastodon is accessed through instance-level API streams. As a decentralized federated network, Mastodon provides open access to public timelines without the restrictive API policies of centralized platforms.

Nextdoor public posts are collected via structured web scraping of neighborhood feeds. Nextdoor provides extremely hyperlocal signals at the street and block level, making it the highest-resolution geographic source in the pipeline.

Telegram public emergency channels and local news channels are ingested using the Telethon Python library. Many local emergency groups, neighborhood channels, and breaking news accounts publish real-time incident reports via Telegram.

Facebook Groups public post content is accessed via the Graph API for public group posts. Large local community groups frequently serve as informal crisis reporting forums.

B. Official Government and Emergency APIs

National Weather Service (NWS) provides a free REST API at weather.gov delivering real-time weather alerts, severe weather warnings, and flood advisories by geographic location. NWS alerts serve as official corroboration signals that the LLM agent uses to increase the severity score of social media clusters reporting weather-related events.

USGS Earthquake Hazards API provides free real-time earthquake data with sub-minute latency. USGS signals cross-validate social posts reporting shaking, building damage, or loud rumbling noises.

FEMA OpenFEMA API provides free access to historical disaster declaration records and flood zone data. These are used primarily for labeling training data and providing geographic context on historically high-risk areas.

Waze Connected Citizens Program makes real-time accident and road closure data available through a public data feed, providing additional incident corroboration for traffic-related crisis signals.

PulsePoint exposes live 911 dispatch incident data in participating jurisdictions through an API reverse-engineered by the developer community. PulsePoint data provides near-ground-truth labels for active incidents, enabling both real-time corroboration and offline evaluation.

C. Crowdsourced and Citizen Platforms

Ushahidi public datasets from past global crisis deployments provide labeled incident reports from real-world emergencies. These datasets supplement the Twitter-centric training data with non-Twitter crisis signal examples.

Citizen App public incident feeds are accessed through web scraping of the public-facing incident map. Citizen posts are directly tied to 911 dispatch data and provide strong ground-truth labels for model evaluation.

GDELT Project monitors global news media in near real-time and publishes free bulk data and streaming API access. GDELT cross-references local news signals at massive scale, providing a high-recall supplementary source for identifying events that have been picked up by local news but not yet widely posted on social media.

Patch.com RSS feeds provide hyperlocal news by locality. Patch covers fires, accidents, power outages, and other incidents in suburban communities at a geographic granularity well-matched to CrisisEcho's Northern Virginia focus.

NewsAPI and **MediaStack** free-tier REST APIs aggregate news sources filterable by location and keyword, supporting both near-real-time signal collection and offline ground-truth labeling of crisis events.

D. Labeled Research Datasets for Training and Evaluation

The following publicly available datasets are used for model training, fine-tuning, and offline evaluation:

- **HumAID** [3]: 77,000 tweets across 19 disaster events, annotated for crisis type, informativeness, and humanitarian category.
- **CrisisNLP** [1]: Multi-event annotated Twitter dataset across dozens of real-world crises with fine-grained labels.
- **CrisisLex** [2]: Curated lexicon plus labeled tweet collections from multiple crisis events.
- **Kaggle Disaster Tweets**: Binary relevance-labeled tweets from a widely used NLP competition dataset.
- **CrisisMMD**: Multimodal dataset combining text and images for crisis classification tasks across four annotation dimensions.
- **FireFlood Dataset**: Labeled social media posts specific to fire and flood events, providing event-type-specific training examples.
- **ISCRAM Datasets**: Multiple labeled corpora from the Information Systems for Crisis Response and Management conference proceedings.

E. Data Limitations

Potential limitations of the described sources include the following. Nextdoor and Facebook scraping may be blocked by platform anti-scraping measures, requiring fallback to open-platform alternatives. Fewer than 2% of Twitter posts carry GPS coordinates, requiring text-based geocoding with inherent uncertainty. PSM and BSM signals from connected infrastructure are not available in the Northern Virginia region at the time of writing. Patch.com and GDELT may have latency of several minutes before news articles are indexed, limiting their utility as real-time signals compared to social media streams. Despite these limitations, the combined source portfolio provides sufficient signal density and redundancy for reliable early-warning detection across the Northern Virginia and DC metro study area.

VI. Proposed Approaches

A. Multi-Source Ingestion with Kafka

All data sources feed into a Kafka message broker that decouples ingestion from processing. Separate Kafka topics are maintained for each source class: `social_raw` for all social media platforms, `official_alerts` for NWS, USGS, FEMA, and PulsePoint feeds, `news_feed` for GDELT, Patch, and NewsAPI, and `citizen_reports` for Ushahidi and Citizen App. Go consumers read from each topic and trigger the Python preprocessing service via internal REST calls. For batch historical datasets (HumAID, CrisisNLP), a Go replay worker simulates streaming at a configurable throughput rate to enable end-to-end pipeline testing.

B. Semantic Pre-filtering with DistilBERT

Before any post enters the vector index or LLM pipeline, a lightweight DistilBERT classifier fine-tuned on HumAID performs binary relevance classification. This pre-filter discards approximately 70% of incoming posts as off-topic, reducing the volume of embeddings generated and LLM API calls required. Only posts passing the relevance threshold proceed to geocoding and embedding.

C. Hybrid Geospatial and Vector Retrieval

For each periodic trigger (every 60 seconds or on significant volume spike), the retrieval module issues a hybrid query combining three concurrent lookups. First, a Pinecone ANN search retrieves the top 50 semantically similar posts from the last two hours filtered by geographic bounding box. Second, a MongoDB `$near` query retrieves all posts within a configurable radius of the trigger centroid that may have low semantic similarity but share geographic proximity. Third, any active official alerts from NWS, USGS, or PulsePoint in the same area are retrieved from MongoDB and injected as high-authority context documents. The three result sets are merged and de-duplicated before being passed to the LLM agent.

D. Three-Step LLM Agent Chain

The core reasoning pipeline is implemented as a three-step LangChain agent chain. In step one, the agent receives the merged retrieval results as context and performs event clustering, identifying distinct real-world events being described across the posts and producing a structured JSON output specifying event type, location description, estimated time, contributing post identifiers, and a confidence score. In step two, the agent assesses cluster severity on a 1–5 scale considering language urgency, count of independent reporters, cross-platform corroboration, and presence of any official corroborating alert. The agent outputs both a numeric severity score and a two-sentence rationale. In step three, the agent generates a two-to-three sentence plain-language alert for public consumption, ensuring no personally identifiable information appears in the output and appending a source platform attribution list.

E. Severity Thresholding and Notification

Alerts are only published to the notification system when the cluster satisfies three conditions simultaneously: the LLM-assigned severity score is 3 or higher on the 1–5 scale, the cluster contains at least three independent post contributors from at least two distinct platforms, and the cluster centroid geocoding confidence exceeds a minimum threshold. This multi-condition filter substantially reduces notification fatigue compared to single-threshold systems.

VII. System Architecture

A. Architecture Overview

CrisisEcho is structured as a five-layer pipeline. Layer one handles multi-source ingestion through Kafka. Layer two performs NLP preprocessing and enrichment in Python. Layer three provides hybrid storage across MongoDB Atlas, Pinecone, and Redis. Layer four executes the RAG-powered LLM reasoning agent. Layer five delivers alerts and interactive visualizations through the Go API and React frontend.

The system architecture diagram is provided below.

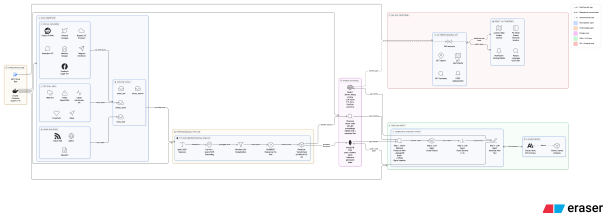

Fig. 1. CrisisEcho System Architecture

B. Layer 1: Ingestion and Streaming

Platform-specific ingestion workers are implemented in Python using Tweepy (Twitter/X), PRAW (Reddit), the AT Protocol Python SDK (Bluesky), Telethon (Telegram), and standard HTTP clients (NWS, USGS, GDELT, NewsAPI, Patch RSS). Each worker publishes raw post objects as JSON to the appropriate Kafka topic. Kafka consumer groups in Go handle backpressure and provide at-least-once delivery guarantees. Dead-letter queues capture malformed or oversized messages for offline inspection.

C. Layer 2: Preprocessing and Enrichment

A Python microservice consumes from the `social_raw` and related Kafka topics and applies the following sequential steps. Text cleaning uses a spaCy pipeline with custom components for URL removal, emoji normalization (optionally retained as sentiment signal), and whitespace normalization. Geocoding applies Carmen as the primary extractor followed by spaCy NER for location entity identification. If neither method yields a location, the post is tagged as `location:unresolved` and held for batch geo-inference using nearby geotagged posts as geographic priors. Near-duplicate detection uses MinHash LSH via the datasketch library with a Jaccard similarity threshold of 0.85 and a five-minute deduplication window. Relevance prefiltering applies the DistilBERT classifier. Embedding generation uses the `all-MiniLM-L6-v2` Sentence-Transformers model to produce 384-dimensional dense vectors, configurable to `all-mpnet-base-v2` for higher accuracy at the cost of throughput.

D. Layer 3: Hybrid Storage

1) MongoDB Atlas (Primary Document and Geospatial Store): MongoDB Atlas free tier M0 serves as the primary document and geospatial store. All post documents, cluster records, and alert records are stored in MongoDB. The `posts` collection uses a 2dsphere index on the GeoJSON `location` field to enable efficient geospatial queries. The `clusters` collection stores both a centroid point and a convex hull polygon for each detected crisis cluster. The `alerts` collection maintains a full audit trail including which posts contributed to each alert and the LLM-generated rationale.

2) Pinecone (Vector Index): Pinecone's hosted free Starter plan provides a persistent vector index with support for up to 100,000 vectors and one index at no cost. Unlike Qdrant Cloud or Weaviate Cloud sandbox tiers, Pinecone's Starter plan does not expire due to inactivity, ensuring that embeddings accumulated during Phase 1 remain accessible throughout the project duration. The Pinecone index stores one vector per processed post with payload metadata fields for post identifier, timestamp, latitude, longitude, source platform, and crisis type, enabling combined ANN search plus metadata filtering in a single query. The Python `pinecone-client` library handles all vector operations; the Go API queries Pinecone via its REST API at inference time.

3) Redis (Hot Cache and Pub/Sub): Redis stores the sliding window of the most recent 30 minutes of processed posts as a sorted set indexed by timestamp. LLM-generated cluster summaries are cached with a five-minute TTL to prevent redundant API calls for geographically overlapping retrieval queries. The Redis Pub/Sub channel `alerts:live` carries new alert events from the Python agent to the Go API server, which relays them to connected React clients over WebSocket.

E. Layer 4: RAG Pipeline and LLM Agent

The RAG pipeline is implemented in Python using LangChain for agent orchestration and LlamaIndex for document loading and context window management. The LLM backend is Claude claude-haiku-4-5 (primary, optimized for speed and cost) with Ollama-hosted Llama3 as a configurable free fallback for offline or budget-constrained operation. The agent is stateless; all relevant context is included in each API call as retrieved documents. New cluster documents are written back to MongoDB upon successful completion of all three agent steps, and a Redis Pub/Sub message is published to trigger downstream alert delivery.

F. Layer 5: Go API and React Frontend

The Go backend uses the Fiber HTTP framework and exposes the following REST endpoints. `GET /api/hotspots` accepts latitude, longitude, and radius parameters and returns all active crisis clusters within the specified area. `GET /api/pin` returns detailed cluster information for a user-dropped map pin including the LLM summary, contributing posts, timeline, and source breakdown. `POST /api/subscribe` stores user notification preferences (location, crisis type filters, radius). `GET /api/query` accepts a natural-language text query, routes it to the LLM agent with a specialized digest prompt, and returns a structured response. `GET /ws/alerts` is a WebSocket endpoint that streams new alert events to connected clients using Redis Pub/Sub as the backend message bus.

The React frontend uses TypeScript and react-leaflet for the interactive map. Crisis hotspots are rendered as circle markers sized proportionally to severity score and colored by crisis type (red for life-safety, orange for infrastructure, yellow for traffic, green for positive events in the stretch goal). A slide-up pin detail drawer shows the LLM summary, a Recharts timeline of post volume over time, anonymized post excerpts with source labels, and cross-platform corroboration indicators. A

WebSocket hook subscribes to `/ws/alerts` and renders toast notifications for new high-severity events. A natural-language query bar allows users to type questions such as "What is happening near Dulles right now?" and receive an LLM-generated digest.

G. Infrastructure and Deployment

All services are defined in a single Docker Compose file for local development and end-to-end testing. Production deployment targets GCP Cloud Run, which provides serverless container hosting with a generous free tier. MongoDB Atlas and Pinecone are externally hosted and require no infrastructure management. The Go API and Python preprocessing service are containerized separately, enabling independent scaling. The React frontend is built to a static bundle and hosted on Firebase Hosting or GCP Cloud Storage.

VIII. Database Schema Design

A. MongoDB: posts Collection

Each document in the `posts` collection contains the following fields. The `_id` field is a MongoDB ObjectId. The `post_id` field is a string in the format `platform:native_id` (e.g., `twitter:1234567890`). The `source` field identifies the platform. The `raw_text` and `cleaned_text` fields store the original and preprocessed content respectively. The `timestamp` field is an ISODate. The `location` field is a GeoJSON Point object {`type: "Point", coordinates: [lng, lat]`} covered by a 2dsphere index. The `location_confidence` is a float from 0 to 1 indicating geocoding certainty. The `location_source` field identifies whether the location was derived from GPS, Carmen, NER, user profile, or inference. The `embedding_id` field references the corresponding Pinecone vector. The `cluster_id` is a nullable ObjectId referencing the `clusters` collection. The `crisis_type` field is a nullable categorical string from the set: flood, shooting, outage, fire, accident, earthquake, or null. The `severity_score` is an integer from 1 to 5. The `is_relevant` field is a boolean output of the DistilBERT pre-filter. The `metadata` subdocument stores a hashed username and platform-specific metadata.

B. MongoDB: clusters Collection

Each document in the `clusters` collection contains the following fields. The `centroid` field is a GeoJSON Point covered by a 2dsphere index. The `affected_area` field is a GeoJSON Polygon representing the convex hull of all contributing post locations. The `crisis_type`, `severity`, and `severity_rationale` fields store the LLM agent outputs from step two of the reasoning chain. The `summary` field stores the step-three alert text. The `post_count` and `post_ids` fields track cluster membership. The `sources` array lists all contributing platforms. The `official_corroboration` boolean indicates whether any NWS, USGS, or PulsePoint signal was present in the retrieval context. The `start_time`, `last_updated`, and `status` fields (active, resolved, or stale) track cluster lifecycle.

C. Pinecone: post_embeddings Index

The Pinecone index stores 384-dimensional float32 vectors generated by `all-MiniLM-L6-v2`. Each vector is associated with a metadata payload containing: `post_id`, `timestamp` (Unix epoch), `lat`, `lng`, `source`, `crisis_type`, and `severity_score`. All metadata fields are indexed for filtering, enabling combined ANN plus metadata queries such as "find the top 50 semantically similar posts from the last two hours within a 50km radius."

IX. RAG Pipeline: Detailed Specification

A. Retrieval Strategy

For each pipeline trigger, the retrieval module issues three concurrent queries. The Pinecone query uses HNSW ANN search to retrieve the top 50 vectors with `timestamp` greater than current time minus 7200 seconds and `lat`/`lng` within the trigger bounding box. The MongoDB query uses `$near` with a configurable radius (default 50km) to retrieve posts with low semantic similarity but high geographic proximity. Official signals from the `official_alerts` Kafka topic that have been persisted to a dedicated MongoDB collection are retrieved using a spatial intersection query. Results are merged, de-duplicated on `post_id`, and ranked by a composite score combining semantic similarity, recency, and source authority.

B. LLM Prompt Templates

The step-one clustering prompt instructs the model: "Given these N social media posts from the past two hours near [location], identify distinct real-world events being described. For each event output: event_type, location_description, estimated_time, contributing_post_ids, and confidence_score as JSON."

The step-two severity prompt instructs the model: "Given this cluster of N posts describing [event_type] near [location], rate severity 1–5 where 1 is unconfirmed minor incident and 5 is confirmed mass casualty or critical infrastructure failure. Consider language urgency, number of independent reporters, and any official corroboration. Output: severity_score (integer) and rationale (two sentences) as JSON."

The step-three alert prompt instructs the model: "Write a two-to-three sentence alert for general public consumption about this severity-[X] [event_type] event near [location]. Be factual, calm, and actionable. Do not include usernames. Append: Sources: [platform list]."

X. Evaluation Methodology

A. Baseline Comparison

CrisisEcho is evaluated against a keyword-based baseline that applies the CrisisLex lexicon to flag posts matching any crisis-related term. The same post corpus, label set, and evaluation split are used for both systems, enabling a direct comparison of precision, recall, and F1.

B. Evaluation Metrics

The primary evaluation metrics are as follows. Precision at K measures the fraction of the K fired alerts that correspond to a verified real-world event in the ground truth, with a target of greater than 0.75. Recall measures the fraction of ground-truth events that CrisisEcho detected with at least one alert, with a target of greater than 0.70. End-to-end latency measures the time from the first social post about an event to the alert appearing on the map, with a target of under 90 seconds. The false positive rate measures the fraction of fired alerts with no corresponding real event, with a target below 0.20. Geocoding accuracy measures the fraction of posts geocoded to the correct city or neighborhood, with a target above 0.80. The F1 improvement over the keyword baseline is expected to exceed 15 percentage points based on comparable systems in the literature.

C. Ground Truth Construction

Ground truth labels are constructed from two sources. Offline evaluation uses the HumAID and CrisisNLP test splits with their existing event labels. Online evaluation during the project period uses PulsePoint dispatch records and NWS active alerts as independent ground truth for Northern Virginia events, enabling evaluation of the full live pipeline.

XI. Challenges and Mitigations

Noisy Unstructured Text. Social media contains typos, sarcasm, memes, and off-topic posts at high volume. The DistilBERT pre-filter removes approximately 70% of noise before LLM processing, and MinHash LSH deduplication removes reposts and retweets.

Sparse Geolocation. The Carmen and spaCy NER cascade with profile location fallback addresses most cases. Posts with no extractable location are held for batch geo-inference using nearby geotagged posts as geographic priors rather than being discarded.

LLM API Cost. The Claude Haiku model is the lowest-cost Anthropic API tier. Posts are batched before each API call. Redis caches cluster summaries for five minutes. Llama3 via Ollama provides a free offline fallback.

Pinecone Free Tier Limits. The Starter plan provides 100,000 vectors. Embeddings older than 72 hours are deleted via a scheduled cleanup worker, maintaining headroom within the free tier throughout the project.

Real-Time Latency. Kafka batching, Go concurrency, and Redis caching keep end-to-end latency under the 90-second target. The DistilBERT pre-filter reduces the volume of posts requiring embedding and LLM processing by approximately 70%.

False Positives and Notification Fatigue. The three-condition alert threshold (severity 3 or higher, at least three independent contributors from at least two platforms, minimum geocoding confidence) substantially reduces false positive alerts compared to single-threshold systems.

Privacy and Ethics. All usernames are hashed at ingestion using SHA-256 before storage. No personally identifiable information appears in LLM prompts or alert outputs. Notifications are strictly opt-in. The system is designed for situational awareness and safety, not surveillance.

Platform Access Restrictions. Nextdoor and Facebook scraping may be blocked. Both are classified as supplementary sources. Bluesky, Mastodon, and Reddit provide equivalent open-API coverage as fallback alternatives.

XII. Technology Stack Summary

Table I summarizes the complete technology stack with rationale for each selection.

TABLE I
CrisisEcho Technology Stack

Component	Technology	Rationale
Backend API	Go (Fiber)	High-throughput concurrent alert delivery; goroutines for WebSocket management
AI Pipeline	Python 3.11 + LangChain	Rich NLP/LLM ecosystem; LangChain for multi-step agent orchestration
Frontend	React 18 + TypeScript + Leaflet.js	Component-based UI; react-leaflet for interactive geospatial map
Primary DB	MongoDB Atlas (Free M0)	Document store with native 2dsphere geospatial indexing; no PostGIS needed
Vector DB	Pinecone (Free Starter)	Persistent hosted ANN search; payload-filtered vector queries; no inactivity expiry
Cache / Pub-Sub	Redis 7	Sliding post window; LLM result cache; WebSocket alert relay
Message Queue	Apache Kafka	Decouples ingestion from processing; simulates real-time streaming
NLP	spaCy + Sentence-Transformers + datasketch	Cleaning, NER, embeddings, MinHash LSH deduplication
Geocoding	Carmen + spaCy NER	Cascading text-based location extraction from unstructured posts
LLM Backend	Claude claude-haiku-4-5 / Ollama Llama3	Haiku for speed/cost; Llama3 as free offline fallback
Containers	Docker + Docker Compose	Single-command local dev; reproducible environment
Deployment	GCP Cloud Run	Serverless, pay-per-use, free tier covers prototype traffic
Visualization	Recharts + Leaflet.js	Timeline charts and interactive geospatial map in React

References

[1] M. Imran, P. Mitra, and C. Castillo, "Twitter as a lifeline: Human-annotated Twitter corpora for NLP of crisis-related messages," in *Proc. 10th Int. Conf. Language Resources and Evaluation (LREC)*, 2016, pp. 1638–1643.

[2] A. Olteanu, C. Castillo, F. Diaz, and S. Vieweg, "CrisisLex: A lexicon for collecting and filtering microblogged communications in crises," in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014, pp. 376–385.

[3] F. Alam, F. Sajjad, H. Dalvi, N. Durrani, K. Darwish, and P. Nakov, "HumAID: Human-annotated disaster incidents data from Twitter with deep learning benchmarks," in *Proc. 15th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2021.

[4] S. E. Middleton, L. Middleton, and S. Modafferi, "Real-time crisis mapping of natural disasters using social media," *IEEE Intelligent Systems*, vol. 29, no. 2, pp. 9–17, 2014.

[5] Y. Kryvasheyeu, H. Chen, N. Obradovich, E. Moro, P. Van Hentenryck, J. Fowler, and M. Cebrian, "Rapid assessment of disaster damage using social media activity," *Science Advances*, vol. 2, no. 3, p. e1500779, 2016.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuebler, M. Lewis, W. Yih, T. Rocktaschel, and S. Riedel, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023.

[8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[9] D. T. Nguyen, K. A. Al Mannai, S. Joty, H. Sajjad, M. Imran, and P. Mitra, "Robust classification of crisis-related data on social networks using convolutional neural networks," in *Proc. 11th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2017, pp. 632–635.

[10] M. Dredze, M. Paul, S. Bergsma, and H. Tran, "Carmen: A Twitter geolocation system with applications to public health," in *AAAI Workshop on Expanding the Boundaries of Health Informatics Using AI*, 2013.

[11] MongoDB Inc., "Geospatial Queries," MongoDB Documentation, 2023. [Online]. Available: https://www.mongodb.com/docs/manual/geospatial-queries/

[12] G. Schultz, "Prioritizing safety funding using severity weighted risk scores," *Accident Analysis & Prevention*, 2025.

[13] M. Hossain, M. Abdel-Aty, M. A. Quddus, Y. Muromachi, and S. N. Sadeek, "Real-time crash prediction models: State-of-the-art, design pathways and ubiquitous requirements," *Accident Analysis & Prevention*, vol. 124, pp. 66–84, 2019.

APPENDIX

This proposal used AI assistance for content generation and will use AI assistance in writing code throughout the project. All architectural decisions, specification choices, and evaluation designs were formulated by the project team.

TABLE II
TASK ASSIGNMENT TABLE

Task	Assigned To
Define objectives, KPIs, and evaluation metrics	C. Williams
Draft system architecture and data flow diagram	C. Williams
Set repository structure, naming conventions, meeting cadence	C. Williams
Write Ethics, Privacy, and Anonymization section	C. Williams
Final report writing and figure integration	C. Williams
Ingest all data sources; standardize timestamps and identifiers	A. Sivakumar
Basic data QC (missing rates, ranges, anomalies)	Shiyi Liu
Data dictionary (columns, dtypes, units, sources)	Shiyi Liu
Implement spaCy NLP pipeline and Carmen geocoding	A. Sivakumar
Implement MinHash LSH deduplication	A. Sivakumar
Implement DistilBERT relevance pre-filter fine-tuning	A. Sivakumar
Define MongoDB schema and 2dsphere indexes	S. Zih
Implement Pinecone vector index and embedding pipeline	S. Zih
Build LangChain three-step LLM agent chain	S. Zih
Create Kafka topics and Go consumer workers	A. Sivakumar
Implement hybrid retrieval (Pinecone ANN + MongoDB $near)	S. Zih
Integrate official signal injection (NWS, USGS, PulsePoint)	S. Zih
Build Go Fiber REST API and WebSocket server	S. Zih
Build React frontend map, hotspot markers, pin detail drawer	Ki Hong Park
Implement Redis Pub/Sub for real-time alert relay	S. Zih
Implement user notification settings and subscription logic	Ki Hong Park
Implement natural-language query bar (frontend + API)	Ki Hong Park
Run evaluation suite (Precision@K, Recall, latency, geo accuracy)	Shiyi Liu
Sensitivity analysis of severity threshold parameters	Shiyi Liu
Ablations: compare RAG vs. keyword baseline on CrisisNLP split	Shiyi Liu
Dashboard v2 (legends, tooltips, export PNG/CSV)	Ki Hong Park
Docker Compose integration test (all services, single command)	A. Sivakumar
GCP Cloud Run deployment and Firebase frontend hosting	C. Williams
Repository cleanup and README reproduction guide	A. Sivakumar
Demo preparation: crisis scenario replay and live walkthrough	Ki Hong Park
Final proof (numbering, captions, references)	Shiyi Liu
Package submission (PDF + appendices + code link)	Shiyi Liu

TABLE III
PROJECT SCHEDULE (15 WEEKS)

Week	Task
1	Literature Review and Requirements Finalization: Review social media crisis detection, RAG, and geospatial database literature; finalize project scope, datasets, and technical stack (Go, Python, React, MongoDB, Pinecone, Kafka).
2	Infrastructure Setup: Provision MongoDB Atlas M0 free cluster, Pinecone Starter index, Redis Cloud free tier, and Kafka via Docker Compose; define all collection schemas and indexes.
3	Data Ingestion and Preprocessing: Implement all platform ingestion workers (Tweepy, PRAW, AT Protocol, Telethon, NWS, USGS, GDELT, Patch RSS, NewsAPI); implement spaCy NLP pipeline, Carmen geocoding, and MinHash deduplication; ingest 10,000 labeled posts from HumAID and CrisisNLP as baseline corpus.
4	Embedding Pipeline and Pre-filter: Implement Sentence-Transformers embedding generation; fine-tune DistilBERT relevance classifier on HumAID; write all processed posts and embeddings to MongoDB and Pinecone.
5	RAG Retrieval Module: Implement hybrid retrieval combining Pinecone ANN search with MongoDB $near geospatial queries; integrate official signal injection for NWS, USGS, and PulsePoint alerts.
6	LLM Agent Chain: Build LangChain three-step agent chain (cluster, severity, alert); integrate Claude Haiku API; configure Llama3 Ollama fallback; test end-to-end on replayed HumAID subset.
7	Cluster Persistence and Redis Integration: Write LLM agent outputs to MongoDB clusters collection; implement Redis caching for cluster summaries; implement Pub/Sub publish for new alert events.
8	Go API Development: Build Fiber REST API with all endpoints (/api/hotspots, /api/pin, /api/query, /api/subscribe) and WebSocket /ws/alerts server backed by Redis Pub/Sub.
9	React Frontend Development: Build Leaflet map with hotspot markers, pin detail drawer with Recharts timeline, notification settings modal, and natural-language query bar with WebSocket alert toasts.
10	Evaluation Suite: Run offline evaluation on HumAID and CrisisNLP test splits; compute Precision@K, Recall, F1, geocoding accuracy, and end-to-end latency; compare RAG pipeline against CrisisLex keyword baseline.
11	Stretch Features: Implement positive event detection with green markers; begin multi-language support using multilingual-e5 embeddings for Spanish-language posts in the Northern Virginia area.
12	System Testing and Optimization: Full Docker Compose integration test; address latency bottlenecks; tune severity thresholds; harden edge cases in geocoding pipeline and data ingestion workers.
13	Refinement and Documentation: Adjust LLM prompt templates based on evaluation results; prepare technical documentation, API reference, and data dictionary.
14	Demo Preparation: Prepare demo script simulating a flooding crisis from replayed HumAID data; verify full pipeline fires an alert within 90 seconds; prepare presentation slides and architecture diagram.
15	Finalization: Final report writing; repository cleanup with reproduction README; package submission (PDF, appendices, code link, data dictionary).